

GDA Full E2E Test Map

Date: April 27, 2026

Purpose

This document defines what “full end-to-end testing” means for GDA Command.

The current system has a passing smoke baseline, but that is not the same as a full business-process E2E certification. Smoke tests prove that pages load, critical proxy endpoints respond, and known auth failures are not recurring. Full E2E tests prove that real user journeys work from React click, through the API proxy, through n8n, into Postgres or external systems, and back to the UI.

Current baseline

Already proven:

Area	Status
React sidebar route smoke	23/23 passing
React functional smoke	8/8 passing
React API proxy smoke	10/10 passing
Read-only E2E journeys	5/5 passing
Phase B dry-run E2E checks	2/2 passing
Direct browser-to-n8n webhook calls	0 detected in tested routes
Browser console errors	0 detected
API auth failures	0 detected
React-used n8n workflow smoke group	10/10 passing in local QA-agent run
Retool-used n8n workflow smoke group	4/4 passing in local QA-agent run

Conclusion: Stabilization Phase 1 is complete. Phase A read-only E2E is complete. Phase B now has two safe dry-run workflows passing.

E2E test levels

Level	Name	What it proves	Current status
1	Smoke	Page loads, no console errors, no direct n8n webhook calls	Built
2	API proxy	React-critical <code>/api/...</code> routes respond without auth failure	Built

Level	Name	What it proves	Current status
3	Workflow smoke	n8n workflows respond with HTTP success	Built for selected groups
4	Business journey	A real user action produces the expected business result	Phase A built; Phase B started
5	Data integrity	Postgres writes/updates are correct and reversible/test-safe	First dry-run path built
6	Recovery	Failures are visible, explainable, and actionable	Partial

Critical E2E user journeys

These are the recommended first ten E2E journeys. They are ordered by business value and risk.

Priority	Journey	React area	n8n/API path	Data/system proof	Risk level
1	Launchpad loads executive dashboard	Launchpad	dashboard mega, trends, funnel, opp tracker, daily actions	KPI cards, funnel, top opportunities, deadlines render from live data	Read-only
2	Semantic search returns useful results	Global search / Launchpad	gda-semantic-search	Query returns structured search results and UI displays them	Read-only
3	Opportunity tracker list/search works	Opportunity Tracker	gda-opp-tracker	Opportunities load, filters/search work, no empty false positives	Read-only
4	Save or update an opportunity safely	Opportunity Tracker	gda-save-opp / gda-opp-tracker	Save-opportunity dry-run returns <code>dryRun: true</code> through React proxy	Phase B dry-run passing
5	Generate or load capture plan	Capture Plan	gda-capture-plan	Existing plan loads or dry-run generation returns structured sections	AI/API and possible writes
6	Morning briefing displays latest brief	Morning Brief	gda-morning-briefing	Latest brief appears, no stale auth/data-learn errors	Read-heavy, possible external calls
7	Competitive intel returns landscape/crawl data	Competitive Intel	gda-competitive-intel / crawler outputs	Current landscape/crawl results load and are timestamped	External API-heavy

Priority	Journey	React area	n8n/API path	Data/system proof	Risk level
8	Deep research history and request flow	Deep Research	gda-deep-research-history / gda-deep-research	History loads; new request supports dry-run/preview before paid call	AI/API-heavy
9	Risk register loads and creates safe test risk	Risk Register	gda-risk	Existing risks load; dry-run create returns <code>dryRun: true</code> and a no-write preview	Phase B dry-run passing
10	Platform health explains failures	Platform Health / QA Center	QA agent / platform health	Latest failures are summarized with workflow names and recommended action	Read-only

Required standard for each E2E test

Each E2E test should record:

Field	Meaning
Journey name	Human-readable business process
React route	Page or component under test
User action	Click, search, submit, filter, generate, save
API route	<code>/api/...</code> endpoint called by React
n8n workflow	Workflow expected to run
Database tables	Tables read/written
External APIs	Any paid or external services used
Safe mode	read-only, dry-run, test row, or approval required
Expected result	What a human should see in React
Failure signal	What proves the journey failed
Cleanup	How test data is removed or ignored

Safety model

Do not make every E2E test write real business data.

Recommended safety categories:

Category	Examples	Automation rule
Read-only	Load dashboard, list opps, latest brief	Safe to run daily
Dry-run	Generate capture plan preview, stage promotion preview	Safe if workflow supports <code>dryRun</code>

Category	Examples	Automation rule
Test row	Create a clearly marked QA opportunity/risk/action	Safe only with cleanup or ignored test namespace
Approval required	Send Telegram/email, call paid APIs, update real opportunity stage	Human approval required

Recommended build sequence

Phase A: Button-ready read-only E2E

Build these first because they are safe and useful:

1. Launchpad dashboard E2E
2. Semantic search E2E
3. Opportunity tracker read/search E2E
4. Morning briefing latest E2E
5. Platform health summary E2E

Goal: These can run from a React “Run Health Check” button without fear.

Status: Built and passing as `npm run e2e:readonly`.

Phase B: Dry-run/write-safe E2E

Build these after dry-run/test-row rules are explicit:

1. Save/update test opportunity — built for `gda-save-opp` dry-run and passing as `npm run phaseb:dryrun`
2. Generate capture plan preview
3. Risk register test risk create/update — built for `gda-risk` dry-run create and passing as `npm run phaseb:dryrun`
4. Deep research preview/request validation
5. Competitor intel crawl preview

Goal: These prove business actions work without corrupting production data.

Current Phase B rule: no workflow joins the automated Phase B test suite until it has an explicit `dryRun, mode: "preview"`, or controlled test-row cleanup path.

Phase C: Human-approved E2E

Only after Phase A/B are stable:

1. External send workflows
2. Paid API-heavy workflows
3. Opportunity stage promotion

- 4. Bulk data ingestion
- 5. Production deploy/fix workflows

Goal: Human-in-the-loop approval remains mandatory.

QA Center target

The eventual React QA Center should show:

Widget	Purpose
Overall GDA status	Healthy / Needs Review / Broken
Last run	Timestamp and duration
React baseline	Sidebar, functional, API proxy result
n8n baseline	React-used workflows, failed executions
E2E journeys	Pass/fail by business journey
Latest failures	Workflow name, node, plain-English cause
Recommended action	What to fix next
Approve fix	Human-in-the-loop button for controlled fix agent

First implementation recommendation

Build a new React page:

Platform → QA Center

Start with one button:

Run Health Check

The first version should call a read-only n8n QA agent action that returns:

```
{
  "ok": true,
  "overall": "healthy",
  "react": { "status": "passed" },
  "n8n": { "reactUsedPassed": 10, "reactUsedFailed": 0 },
  "latestFailures": [],
  "recommendations": []
}
```

Then expand it to run the full E2E journeys above.

Bottom line

The project is no longer at “nothing works.” It has a working smoke baseline. The next goal is to convert that baseline into business-level E2E tests and make those tests available from a React QA Center button so Shawn does not need PowerShell or raw JSON.